

## PROJECT REPORT

# Temperature Prediction from Weather Data

*Benchmarking Regression Models*

---

Prepared by

**Sankhasubhra Basu**

# 1. Introduction

---

## 1.1 Project Overview

This project focuses on predicting air temperature using other meteorological readings collected from a weather station in Pune. Weather datasets typically record several parameters at the same time — humidity, air pressure, wind conditions, sunlight hours, UV index, cloud cover, and more — alongside the actual temperature. This project explores how well temperature can be estimated purely from these accompanying weather parameters using machine learning regression models.

Multiple regression algorithms, ranging from simple linear models to advanced ensemble and stacking methods, were trained and compared on the same dataset to identify which approach predicts temperature most accurately and reliably.

## 1.2 Objectives

- Load and clean a large hourly weather dataset, retaining only the meteorologically relevant columns.
- Perform exploratory data analysis (EDA) to understand which weather parameters are most strongly related to temperature.
- Identify and remove features that would cause data leakage — variables that are themselves derived from temperature.
- Train and tune eight different regression models, spanning linear, distance-based, tree-based, kernel-based, and ensemble approaches.
- Compare all models using Root Mean Squared Error (RMSE) on both training and test data to judge accuracy and generalisation. It tells us, on average, how far the predicted temperature is from the actual temperature.
- Recommend the most suitable model for reliable, real-world temperature prediction.

## 1.3 Tools and Libraries Used

The project was implemented in Python, using the following libraries:

- NumPy and Pandas — numerical computation and data manipulation.
- Matplotlib and Seaborn — data visualisation.
- scikit-learn (sklearn) — preprocessing, pipelines, model building, hyperparameter tuning, and evaluation metrics.

## 2. Dataset Description

**Data Acquisition:** Historical hourly weather data for Pune was collected through a Python-based web-scraping process from publicly accessible weather-data webpages. The extracted records were cleaned, structured, and consolidated into the final dataset used for analysis.

The table below describes every column present in the original file:

| #  | Column Name       | Description                                                |
|----|-------------------|------------------------------------------------------------|
| 1  | date_time         | Timestamp of the recorded reading (date and hour)          |
| 2  | maxtempC          | Maximum temperature recorded for that day (°C)             |
| 3  | mintempC          | Minimum temperature recorded for that day (°C)             |
| 4  | totalSnow_cm      | Total snowfall recorded, in centimetres                    |
| 5  | sunHour           | Number of sunshine hours for that day                      |
| 6  | uvIndex           | UV index reading                                           |
| 7  | moon_illumination | Percentage of moon illumination                            |
| 8  | moonrise          | Moonrise time for that day                                 |
| 9  | moonset           | Moonset time for that day                                  |
| 10 | sunrise           | Sunrise time for that day                                  |
| 11 | sunset            | Sunset time for that day                                   |
| 12 | DewPointC         | Dew point temperature (°C)                                 |
| 13 | FeelsLikeC        | 'Feels like' apparent temperature (°C)                     |
| 14 | HeatIndexC        | Heat index (°C)                                            |
| 15 | WindChillC        | Wind chill temperature (°C)                                |
| 16 | WindGustKmph      | Wind gust speed (km/h)                                     |
| 17 | cloudcover        | Cloud cover percentage                                     |
| 18 | humidity          | Relative humidity (%)                                      |
| 19 | precipMM          | Precipitation (mm)                                         |
| 20 | pressure          | Atmospheric pressure                                       |
| 21 | tempC             | Actual temperature at that hour (°C) — the target variable |
| 22 | visibility        | Visibility distance                                        |
| 23 | winddirDegree     | Wind direction (degrees)                                   |
| 24 | windspeedKmph     | Wind speed (km/h)                                          |
| 25 | location          | Name of the location (constant — 'pune' for every row)     |

Since the goal was to predict ‘tempC’ purely from numeric weather-condition features, several of these raw columns were not usable or relevant as model inputs and were dropped before analysis: date\_time, moonrise, moonset, sunrise, and sunset are timestamp/text fields rather than numeric weather readings; location is constant across every row (always 'pune') and therefore carries no predictive information; and maxtempC, mintempC, and FeelsLikeC are daily summary or derived temperature values that are themselves tied too closely to the target variable to be used as independent predictors. After removing these, the following 16 numeric columns were retained for further analysis:

- totalSnow\_cm, sunHour, uvIndex, moon\_illumination, DewPointC, HeatIndexC, WindChillC, WindGustKmph, cloudcover, humidity, precipMM, pressure, tempC, visibility, winddirDegree, windspeedKmph.

### 3. Data Cleaning and Pre-processing

Before analysis and modelling, the dataset was checked and prepared as follows:

1. **Column filtering:** Out of the original columns in the raw CSV, only the 16 numerically meaningful weather columns listed above were retained, dropping identifiers and non-numeric fields like date\_time, moonrise, moonset, sunrise, sunset, and location.
2. **Data type check:** df.info() confirmed all 16 selected columns were numeric (integers or floats) with no non-numeric fields left to encode.
3. **Missing value check:** df.isnull().sum() confirmed there were zero missing values across all 116,136 rows and all 16 columns, so no imputation was required.
4. **Removing a zero-variance feature:** totalSnow\_cm was found to have a NaN (undefined) correlation with temperature — this happens when a column has no variation in its values (i.e., it is constant). Since a constant feature carries no useful information for prediction, this column was dropped.

### 4. Exploratory Data Analysis (EDA)

A correlation matrix was computed across all numerical features to understand which weather parameters relate most strongly to temperature (tempC).

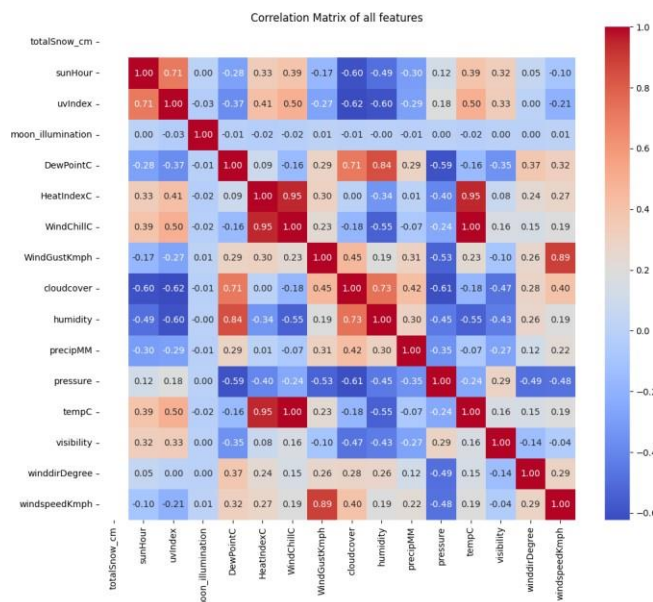


Figure 1: Correlation heatmap of all weather features in the dataset.

## 4.1 Correlation with Temperature

Examining the correlation values specifically against tempC revealed the following:

- **WindChillC** ( $\approx 0.9999$ ) and **HeatIndexC** ( $\approx 0.9466$ ) showed an almost perfect positive correlation with temperature — because both are indices calculated directly from the temperature itself, rather than independent weather measurements.
- **uvIndex** ( $\approx 0.458$ ) and **sunHour** ( $\approx 0.375$ ) showed a moderate positive correlation — more sunlight and higher UV levels are associated with higher temperatures.
- **humidity** ( $\approx -0.530$ ) showed the strongest negative correlation — higher humidity is associated with lower temperatures.
- **pressure** ( $\approx -0.227$ ) and **cloudcover** ( $\approx -0.183$ ) showed weaker negative correlations.

This correlation analysis was critical in guiding the feature-selection decision described in Section 5 — since WindChillC, HeatIndexC, and (later) DewPointC are essentially restatements of temperature rather than independent predictors, using them as input features would let the model 'cheat' rather than genuinely learn from weather conditions.

## 4.2 HeatIndexC vs. Temperature

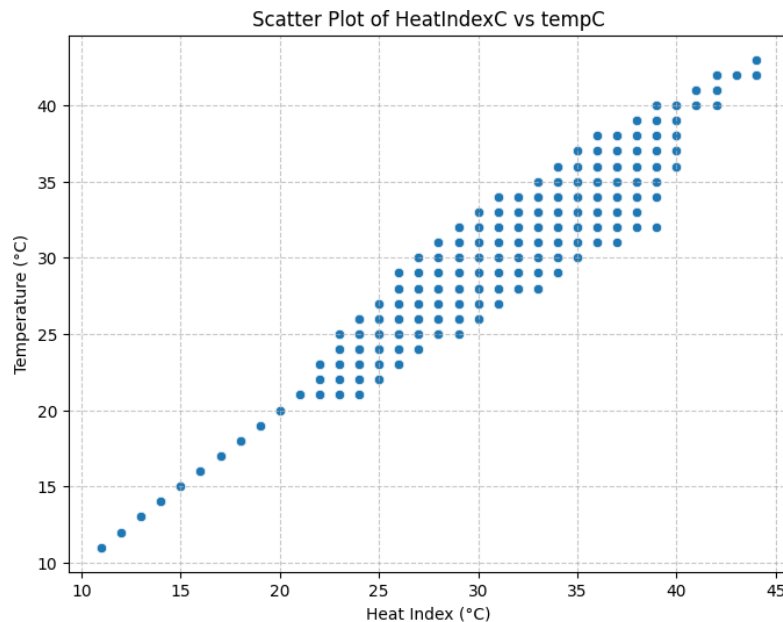


Figure 2: Scatter plot of HeatIndexC against tempC.

The scatter plot shows a near-perfect straight-line relationship between HeatIndexC and tempC, visually confirming that HeatIndexC is directly tied to temperature rather than an independent weather signal.

### 4.3 WindChillC vs. Temperature

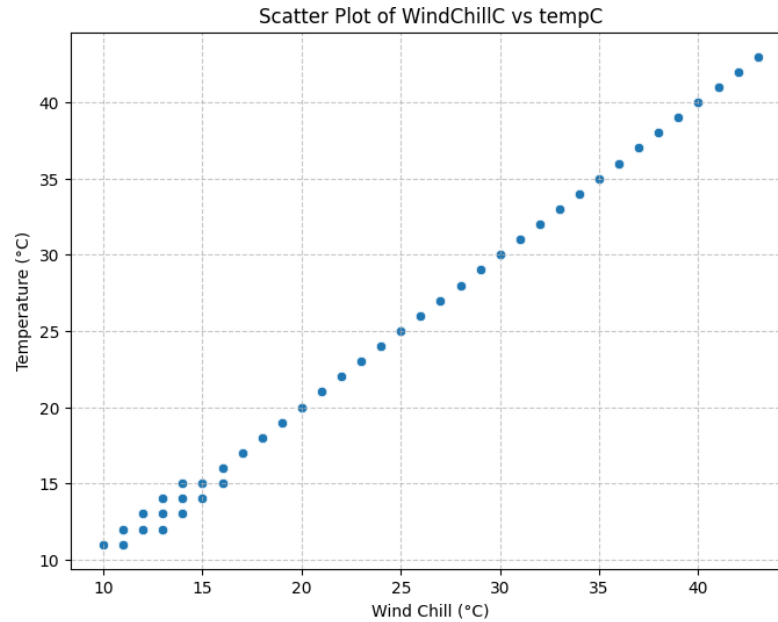


Figure 3: Scatter plot of WindChillC against tempC.

Similarly, WindChillC shows an almost identical one-to-one relationship with tempC, reinforcing that this feature is essentially a re-expression of temperature and must be excluded from the model's inputs to avoid data leakage.

### 4.4 Humidity vs. Temperature

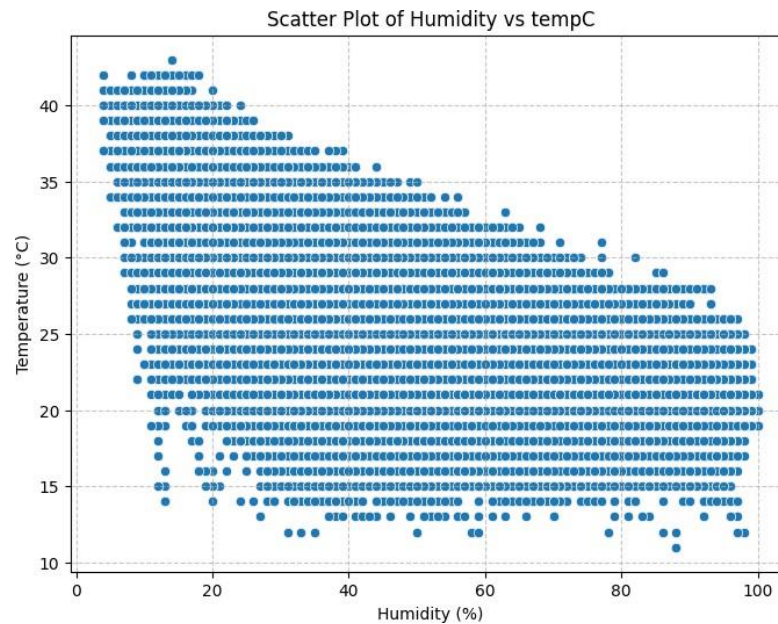


Figure 4: Scatter plot of humidity against tempC.

A visible negative trend appears between humidity and temperature — as humidity rises, temperature tends to fall — though the relationship is more scattered than the previous two, indicating humidity is influenced by other factors as well, not just temperature.

## 4.5 Distribution of Temperature

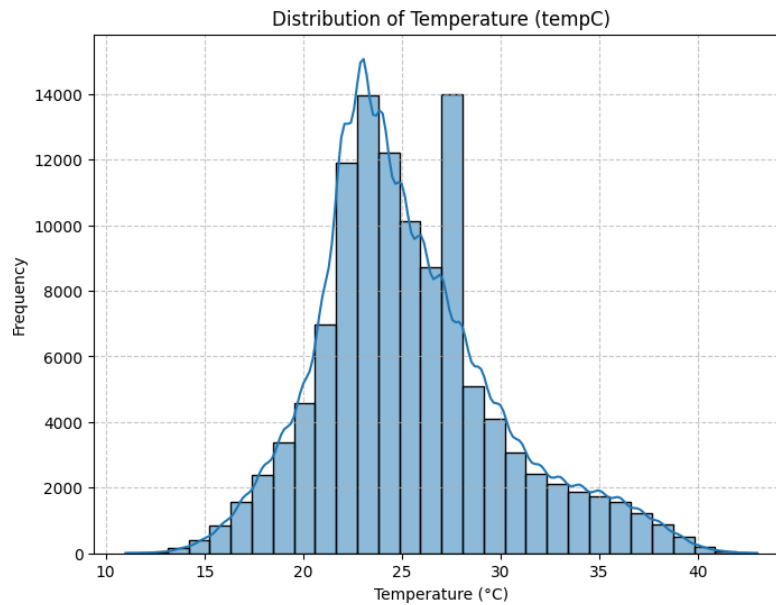


Figure 5: Histogram showing the distribution of tempC values.

The distribution of temperature values is roughly bell-shaped, with most readings concentrated around a central range, and fewer readings at the extreme hot and cold ends.

## 4.6 Temperature across UV Index Levels

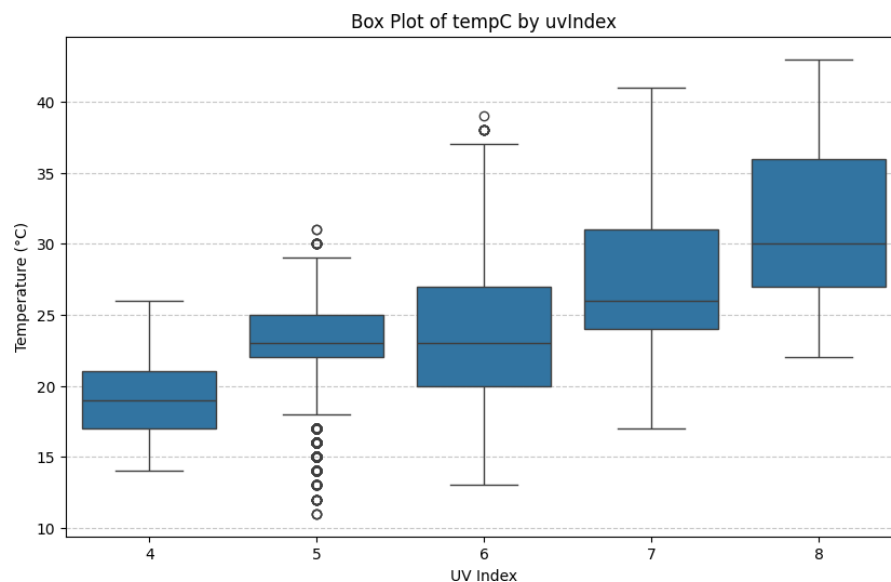


Figure 6: Box plot of tempC grouped by uvIndex.

The box plot shows a clear increasing trend — as UV index rises, both the median temperature and the spread of temperatures increase, confirming the positive correlation observed earlier.

## 5. Feature Selection — Avoiding Data Leakage

---

Based on the correlation analysis and scatter plots, three features — **DewPointC**, **HeatIndexC**, and **WindChillC** — were excluded from the input features used to train the models. Although these are recorded as separate columns in the raw dataset, they are all indices that are mathematically derived from, or extremely tightly linked to, the actual air temperature.

Including such features would cause data leakage: the model would essentially be given a disguised version of the answer as an input, producing unrealistically low error during testing while learning nothing meaningful about how independent weather conditions relate to temperature. Since the goal of the project is to predict temperature from genuinely independent weather signals, these three columns were removed from the feature set.

The final set of input features used for training consists of: **sunHour**, **uvIndex**, **moon\_illumination**, **WindGustKmph**, **cloudcover**, **humidity**, **precipMM**, **pressure**, **visibility**, **winddirDegree**, and **windspeedKmph** — eleven weather parameters that describe atmospheric conditions without directly encoding the temperature itself. The target variable for all models is **tempC**.

## 6. Train-Test Split and Feature Scaling

---

The dataset was split into training and testing sets using an **80:20 ratio**, with a fixed random state to keep the split reproducible across runs. This resulted in roughly **92,900 training rows** and **23,200 testing rows**.

Since some algorithms (such as k-Nearest Neighbours and Support Vector Regression) are sensitive to the scale of input features, all input features were standardised using `StandardScaler`, which was fit only on the training data and then applied to both the training and test sets — this avoids leaking any statistical information from the test set into the scaling process. For the Support Vector Regressor specifically, the target variable (`tempC`) was also scaled, since SVR performs better when both inputs and outputs are on a similar numerical scale; its predictions were converted back to the original temperature scale before evaluation.



## 7. Model Building and Evaluation

Eight regression models were trained and tuned using GridSearchCV with cross-validation, using negative RMSE as the scoring metric so that the best combination of hyperparameters could be automatically selected for each algorithm. For the more computationally expensive models, hyperparameter search was performed on a smaller subset of the training data to keep runtimes practical, after which the best-found configuration was retrained on the full training set. Every model was then evaluated on both the training set and the untouched test set using Root Mean Squared Error (RMSE) — a lower RMSE indicates predictions that are, on average, closer to the true temperature.

### 7.1 Linear Regression

A simple Linear Regression model was tuned over its intercept and positivity-constraint settings. The best configuration used a **fitted intercept** with **no positivity** constraint on coefficients.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 2.6997 |
| Test RMSE  | 2.7032 |

Linear Regression produced very similar train and test error, showing it generalises consistently — but its overall error is the second-highest among all models tested, suggesting that the relationship between the weather features and temperature is not purely linear.

### 7.2 K-Nearest Neighbours (kNN) Regressor

A kNN Regressor was tuned over the number of neighbours and the weighting scheme (uniform vs. distance-based). The best configuration used **10 neighbours** with **distance-based weighting**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 0.0147 |
| Test RMSE  | 1.7869 |

kNN achieved the lowest test RMSE of all eight models, but its training RMSE is almost zero — an extreme gap between training and test error that is a textbook sign of overfitting. In practice, this means the model has essentially memorised the training data rather than learning a generalisable pattern, making its excellent test score less trustworthy for genuinely new, unseen conditions.

### 7.3 Decision Tree Regressor (with Pre-Pruning)

A Decision Tree Regressor was tuned over tree depth and the minimum samples required to split or form a leaf, in order to pre-prune the tree and prevent it from growing into an overfit structure. The best configuration used a **maximum depth of 10**, **minimum samples leaf of 2** and **minimum samples split of 5**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 2.0093 |
| Test RMSE  | 2.1464 |

The pre-pruned Decision Tree achieved a reasonable balance between training and test error, showing that limiting tree depth successfully controlled overfitting while still capturing meaningful, non-linear patterns in the data.

### 7.4 Support Vector Regressor (SVR)

A Support Vector Regressor with an RBF kernel was tuned over its regularisation strength (C) and margin tolerance (epsilon). The best configuration used **C = 10** with an **epsilon of 0.1**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 2.0060 |
| Test RMSE  | 2.0075 |

SVR delivered one of the strongest results of the project, with training and test error nearly identical — indicating excellent generalisation alongside a fairly low overall error.

### 7.5 Random Forest Regressor (Bagging)

A Random Forest Regressor was tuned over the number of trees, tree depth, and minimum split/leaf sizes. The best configuration used **200 trees** with a **maximum depth of 12**. Out-of-bag (OOB) scoring was also enabled, giving an OOB  $R^2$  of approximately 0.840 — an internal validation estimate computed from the data each tree did not see during its own training.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 1.6223 |
| Test RMSE  | 1.8519 |

Random Forest produced the best overall combination of low error and consistent train-test performance among all non-overfitting models, making it the strongest all-round candidate.

## 7.6 Gradient Boosting Regressor (Boosting)

A Gradient Boosting Regressor was tuned over the number of estimators, learning rate, and tree depth. The best configuration used **200 estimators** with a **learning rate of 0.1**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 1.9991 |
| Test RMSE  | 2.0055 |

Gradient Boosting achieved strong, well-balanced performance very close to that of SVR, confirming that boosting a sequence of shallow trees can capture the underlying weather-to-temperature relationship effectively.

## 7.7 AdaBoost Regressor (Boosting)

An AdaBoost Regressor, using shallow Decision Trees as its base estimator, was tuned over the number of estimators, learning rate, and loss function. The best configuration used a **learning rate of 1.0** with a **squared loss function**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 2.8340 |
| Test RMSE  | 2.8523 |

AdaBoost produced the highest error among all eight models. While it did not overfit (train and test error are close), its overall accuracy was the weakest, suggesting its boosting strategy with the chosen base estimator was not well-suited to this dataset.

## 7.8 Stacking Regressor (Heterogeneous Ensemble)

A Stacking Regressor was built by combining three different base models — a scaled Linear Regression, a shallow Decision Tree, and a Gradient Boosting model — and feeding their predictions into a Ridge Regression as the final meta-model. The regularisation strength of this final Ridge model was tuned, with the **best alpha value to be 0.01**.

| Metric     | Value  |
|------------|--------|
| Train RMSE | 2.1716 |
| Test RMSE  | 2.1650 |

The Stacking Regressor produced balanced, stable results, but did not outperform the individually stronger models such as Random Forest, SVR, or Gradient Boosting — indicating that, for this dataset, combining these particular base models did not add extra predictive advantage over using the best single model directly.

## 8. Model Performance Comparison

The train and test RMSE of all eight models were plotted together for direct visual comparison.

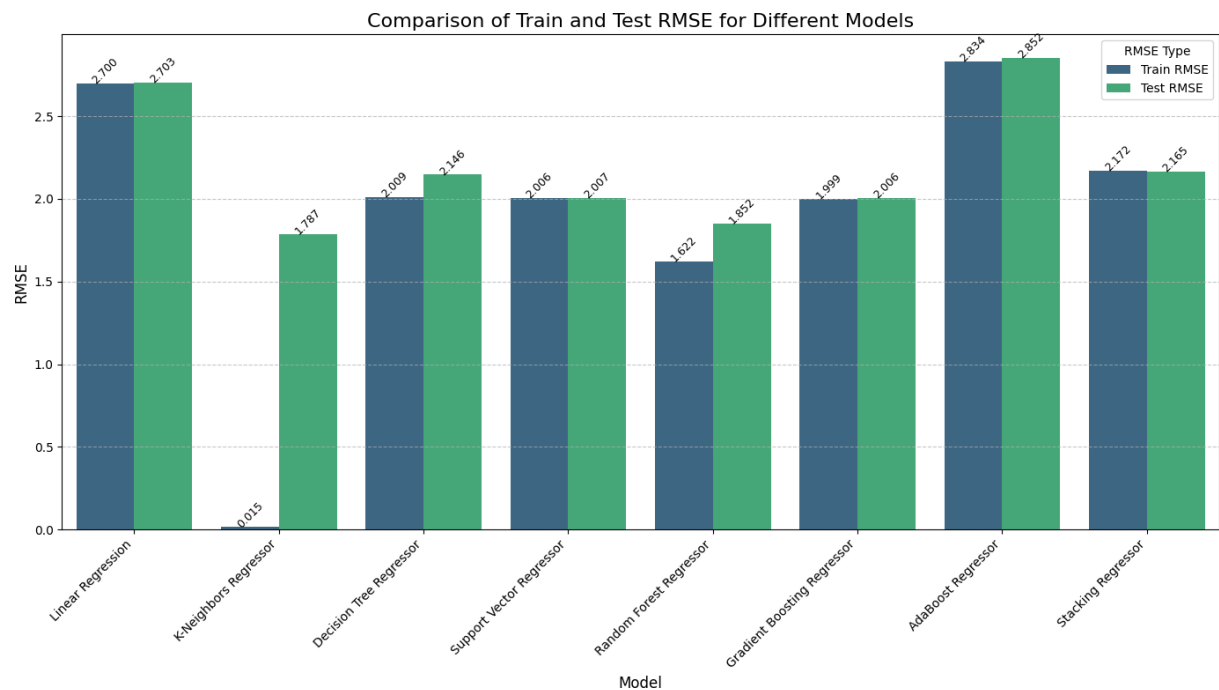


Figure 7: Comparison of train and test RMSE across all eight regression models.

| Model                       | Train RMSE | Test RMSE |
|-----------------------------|------------|-----------|
| Linear Regression           | 2.6997     | 2.7032    |
| K-Neighbors Regressor       | 0.0147     | 1.7869    |
| Decision Tree Regressor     | 2.0093     | 2.1464    |
| Support Vector Regressor    | 2.0060     | 2.0075    |
| Random Forest Regressor     | 1.6223     | 1.8519    |
| Gradient Boosting Regressor | 1.9991     | 2.0055    |
| AdaBoost Regressor          | 2.8340     | 2.8523    |
| Stacking Regressor          | 2.1716     | 2.1650    |

Ranking the models by test RMSE alone; **kNN (1.787)** and **Random Forest (1.852)** are the two strongest performers, followed closely by **SVR (2.008)** and **Gradient Boosting (2.006)**. **Decision Tree (2.146)** and **Stacking (2.165)** fall into a mid-tier group, while **Linear Regression (2.703)** and **AdaBoost (2.852)** perform the weakest.

However, test RMSE alone does not tell the full story. kNN's near-zero training RMSE (0.015) against a test RMSE of 1.787 is a large enough gap to indicate severe overfitting, meaning it has memorised the training data rather than learned a generalisable relationship. Random Forest, by contrast, has a much smaller train-test gap (1.622 vs. 1.852), showing that its low error reflects genuine learning rather than memorisation. **Thus, for real-world deployment and reliable temperature prediction, Random Forest Regressor is the most suitable choice among the models tested.**

## 9. Conclusion and Recommendation

---

This project implemented and compared eight regression models — Linear Regression, k-Nearest Neighbours, Decision Tree, Support Vector Regression, Random Forest, Gradient Boosting, AdaBoost, and a Stacking ensemble — to predict temperature from a set of independent weather parameters, after carefully identifying and removing features that would have caused data leakage.

Considering both accuracy and generalisation together, the **Random Forest Regressor** is recommended as the most suitable model for reliable temperature prediction. It combines one of the **lowest test RMSE values (1.852)** with a **small, consistent gap between training and test error (0.230)**, showing that it captures genuine patterns in the weather data rather than overfitting.

Although kNN achieved a marginally lower test RMSE, its extreme overfitting makes it a riskier and should be interpreted with caution due for new, unseen weather conditions. Random Forest also has the added benefit of being naturally robust to noisy data and outliers, and can still provide some interpretability through feature importance scores.

## 10. Limitations and Scope for Future Improvement

---

### 10.1 Not a True Time-Based Forecast

The current models predict temperature using other weather readings taken at the same point in time, rather than using past readings to predict a future temperature. A genuine forecasting version of this project would require a chronological train-test split (training on earlier time periods and testing on later ones) along with lagged features — for example, using the previous few hours' or days' readings as inputs — so the model learns to project forward in time rather than estimate a simultaneous value.

### 10.2 Hyperparameter Search on Subsets

For computationally expensive models (kNN, Decision Tree, SVR, Random Forest, Gradient Boosting, AdaBoost, and Stacking), hyperparameter tuning was performed on a smaller subset of the training data to keep runtimes practical. Running the search on the full training set, or using randomized/Bayesian search techniques, could potentially find even better-performing configurations.

### 10.3 Additional Feature Engineering

Further engineered features — such as interaction terms between humidity and pressure, or seasonal/time-of-day indicators — could help capture additional patterns and further reduce prediction error.

### 10.4 Model Deployment and Monitoring

The final model could be deployed as a simple prediction API and monitored over time to check that its accuracy remains stable as weather patterns and seasons change, retraining periodically as new data becomes available.